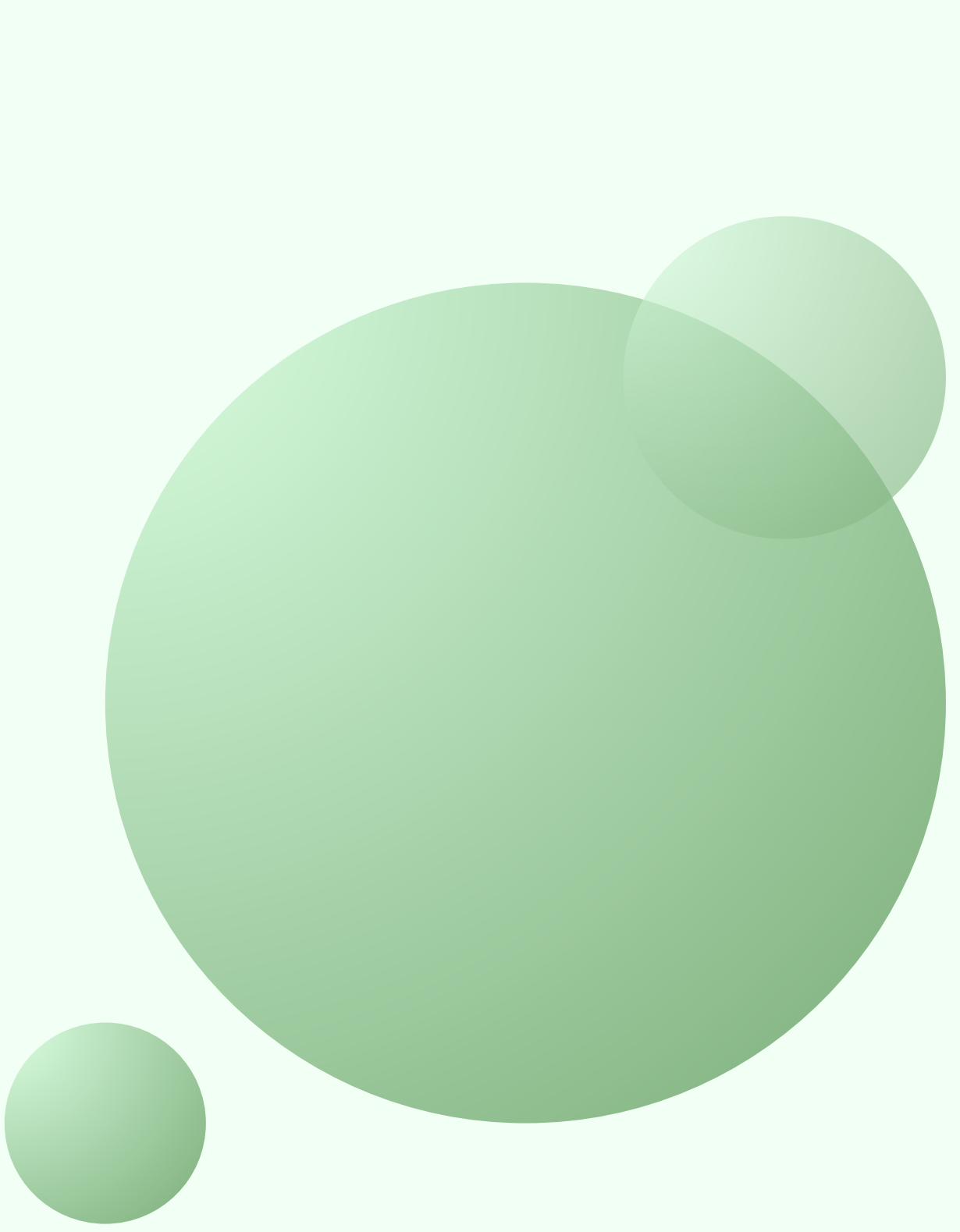




ENK **AGENTS FOR DATA QUALITY**

Presented By :

Emanuele Aicardi (814361), Nurkhanym Ziyabek (810081), Kamila Dochshanova (809891)



OUR **AGENDA**

Results

04

Conclusion

05

Streamlit Demo

06

Introduction

01

Methods

02

Experimental Design

03

INTRODUCTION

NoiPA is the digital platform

of the Italian Ministry of Economy and Finance that manages salaries, timesheets, and tax and social security obligations for employees of the Italian public sector. It periodically receives datasets from heterogeneous sources containing demographic, economic, and tax data. Currently, validation of these datasets is manual or nonexistent, making the process error-prone and time consuming.

This project builds a multi-agent system

that receives a raw CSV dataset, automatically detects quality issues, fixes them and produces a structured quality report including anomalies, correction suggestions, and a final reliability score. The system was tested on three synthetic datasets and two real NoiPA datasets (spesa.csv and attivazioniCessazioni.csv).

METHODS

The system is composed of five specialized agents, each responsible for a specific type of data quality check, orchestrated by a central "run_pipeline" function.

Schema Agent - validates column naming conventions (special characters, leading digits, whitespace) and detects mixed data types within columns

Completeness Agent - identifies missing or placeholder values to calculate completeness scores and flag nearly empty columns for removal

Consistency Agent - ensures logical consistency between fields, uniform formatting within columns and the identification of exact or near-duplicate records

Anomaly Agent - performs outlier detection on numerical columns and highlights rare or unexpected categories to ensure data consistency

Remediation Agent - aggregates findings from all agents, generates concrete actionable suggestions for each issue and computes a final weighted reliability score

EXPERIMENTAL DESIGN

We ran the pipeline on several datasets to validate the system across different levels of data quality and different real-world structures

Synthetic clean dataset

- **Purpose:** verify that the pipeline correctly identifies minor issues in a nearly clean dataset and produces a high reliability score
- **Baseline:** manual inspection of the dataset
- **Metrics:** reliability score, per-agent scores, number of issues detected, number of suggestions generated

Synthetic messy dataset

- **Purpose:** verify that the pipeline correctly identifies severe quality problems (type mismatches, many missing values, extreme outliers) and produces a significantly lower reliability score than Dataset 1
- **Baseline:** manual inspection of the dataset
- **Metrics:** reliability score, per-agent scores, number of issues detected, number of suggestions generated

Synthetic large dataset

- **Purpose:** validate the scalability and robustness of our multi-agent system using a larger, controlled synthetic dataset
- **Baseline:** manual inspection of the dataset
- **Metrics:** reliability score, per-agent scores, number of issues detected, number of suggestions generated



Real NoiPA dataset - Spesa

- **Purpose:** test the pipeline on actual NoiPA payroll and tax spending data to verify it scales to real-world inputs and catches genuine data quality issues
- **Baseline:** synthetic dataset results
- **Metrics:** reliability score, per-agent scores, number of issues detected, number of suggestions generated, plus number of real duplicate rows and outliers detected

Real NoiPA dataset - Attivazioni e Cessazioni

- **Purpose:** verify the pipeline generalises across datasets with a structurally different set of columns (employee activations and terminations per ministry and region)
- **Baseline:** Dataset 4 results
- **Metrics:** reliability score, per-agent scores, number of issues detected, number of suggestions generated, plus number of real duplicate rows and outliers detected

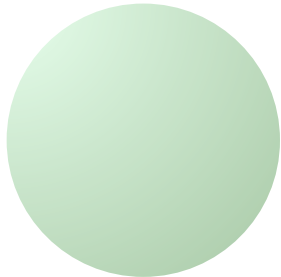
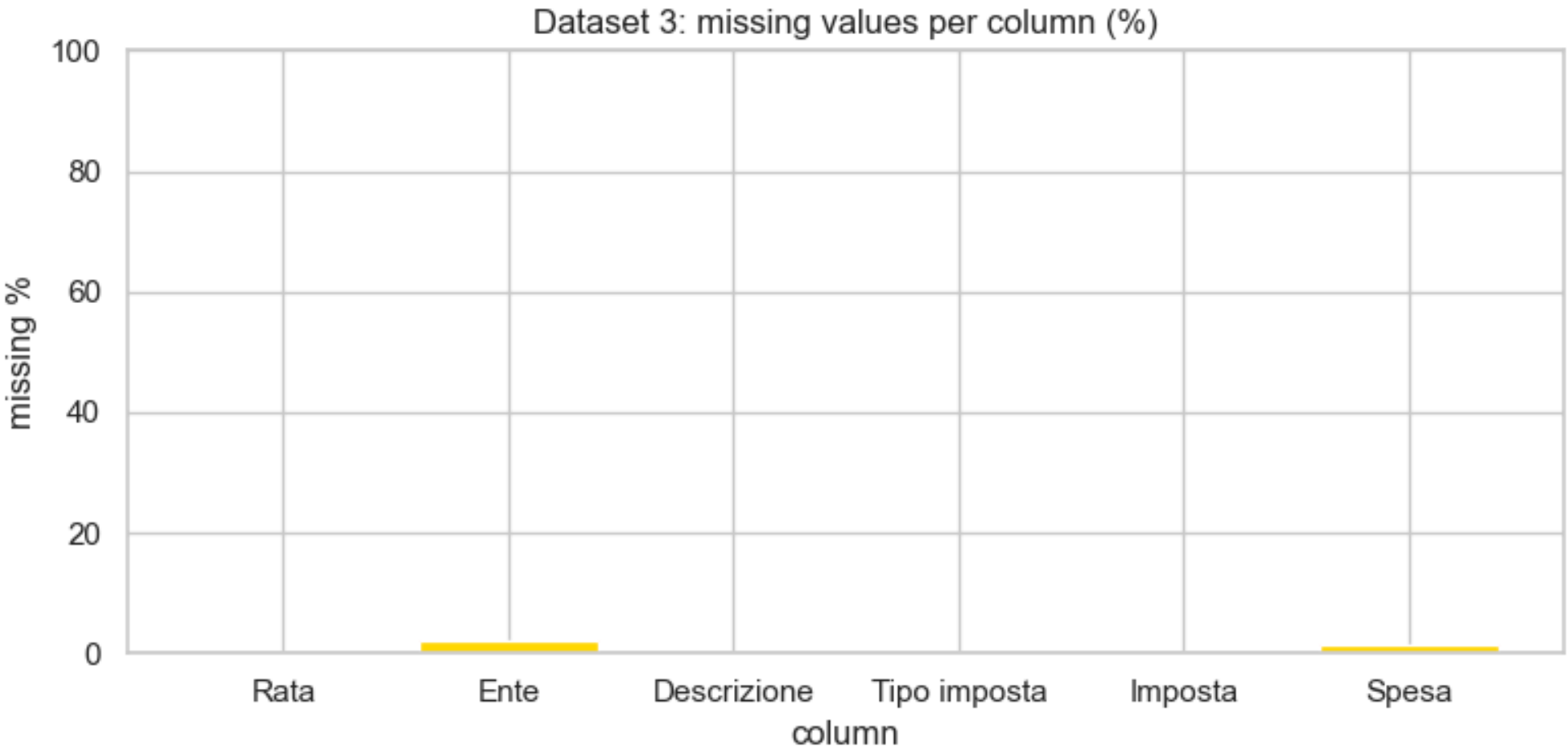
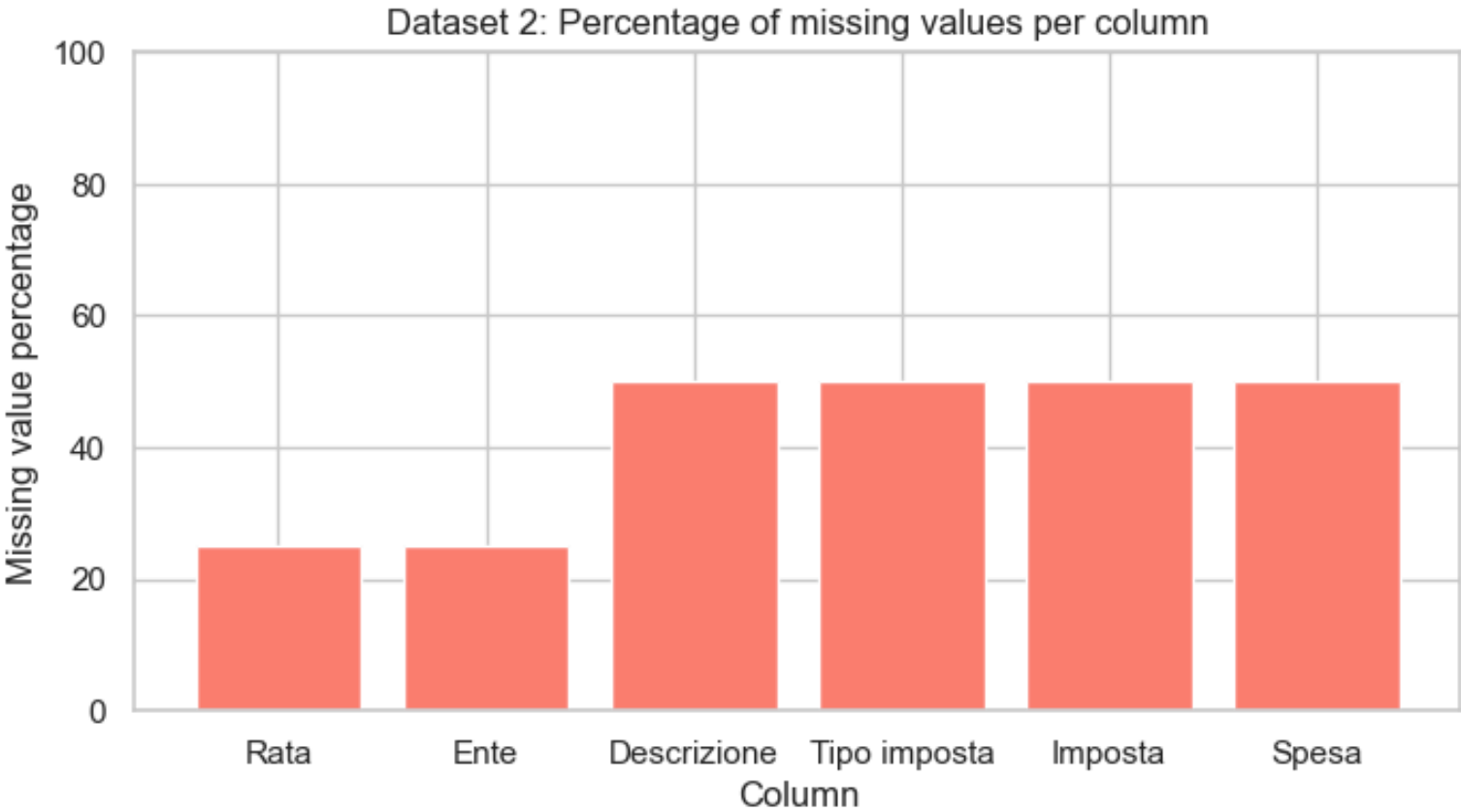
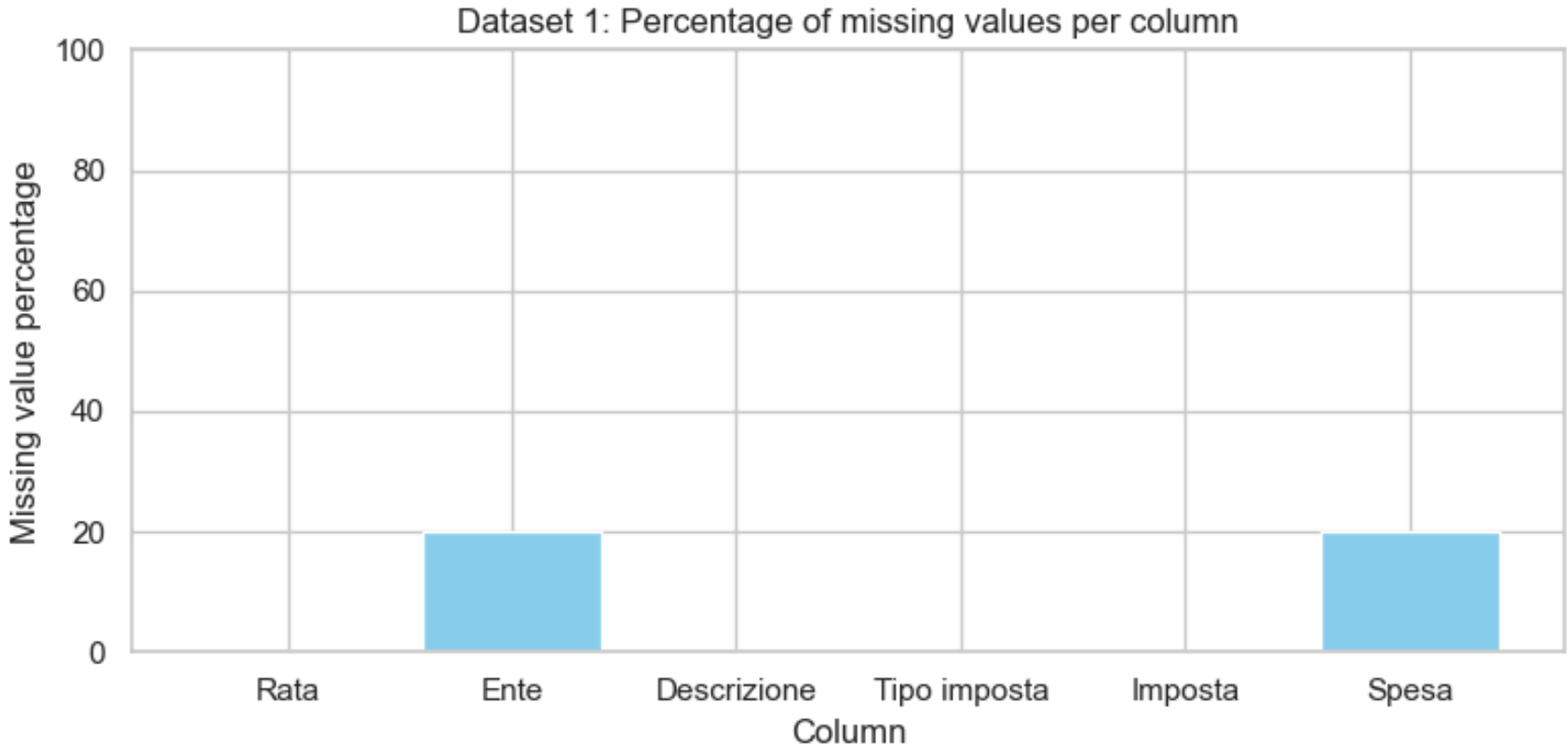


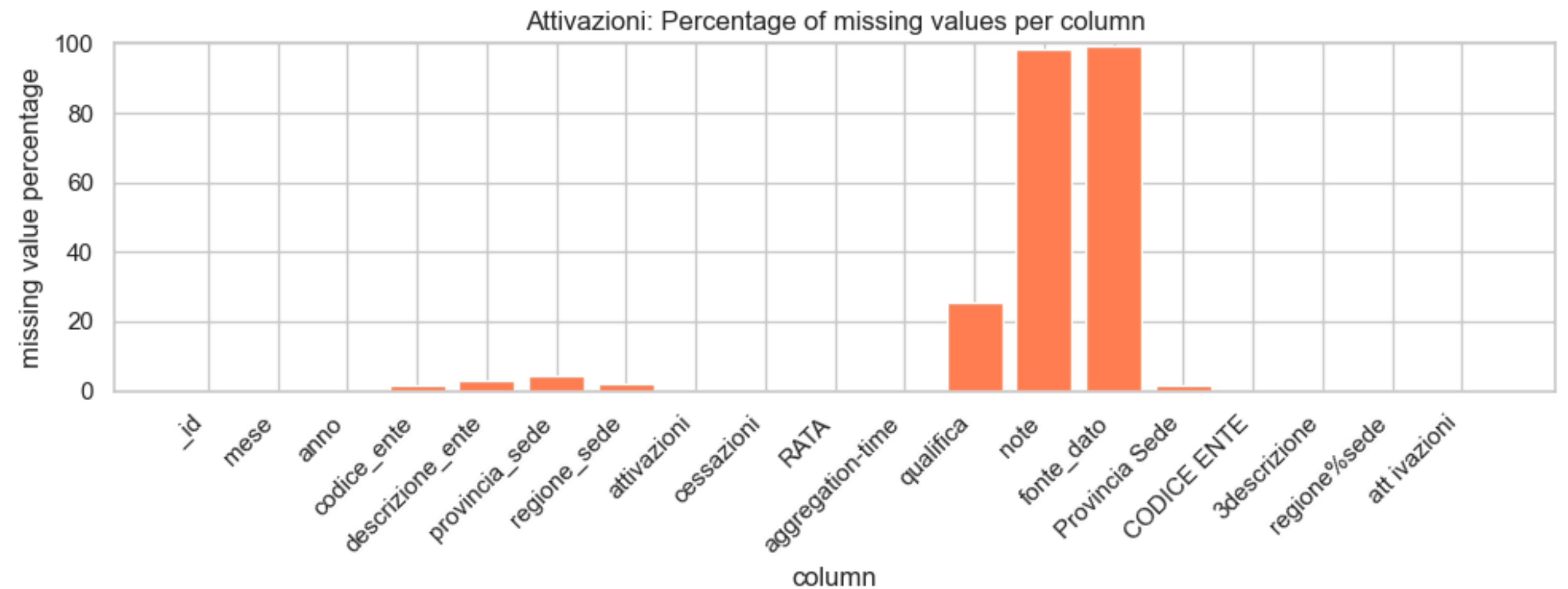
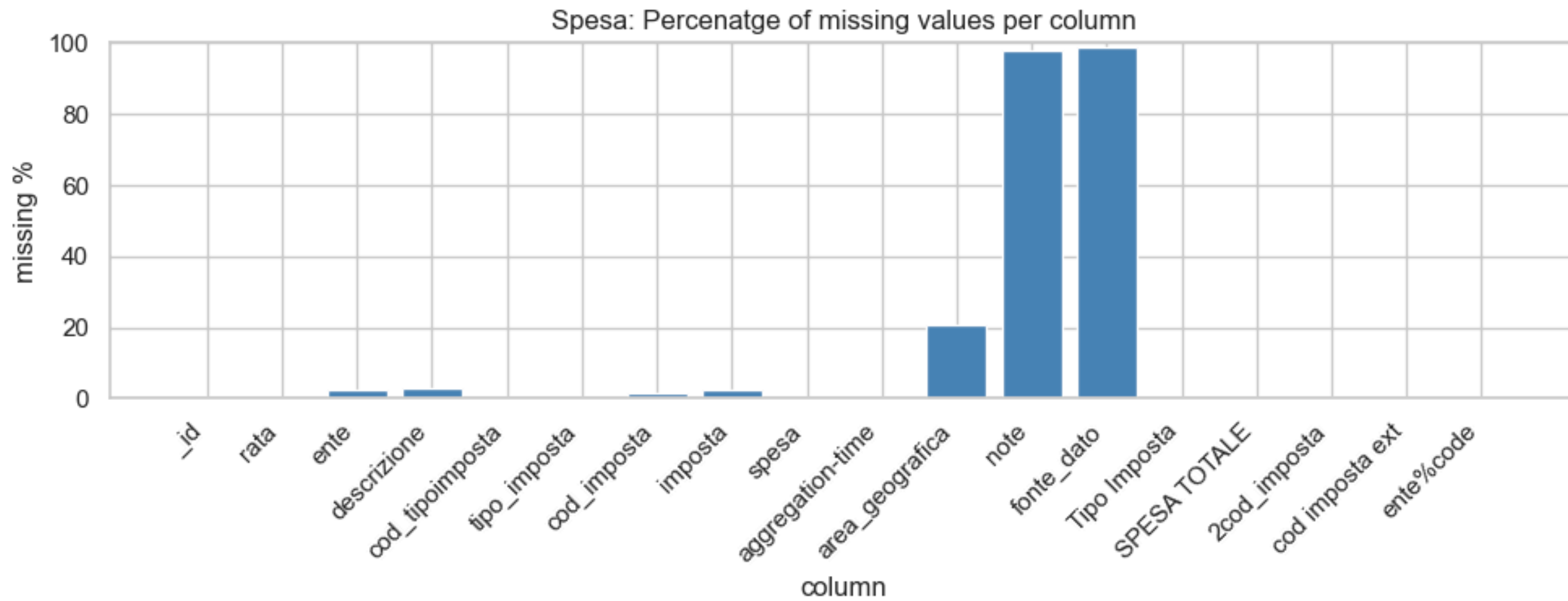
RESULTS

Dataset	Schema	Schema	Schema	Schema	Schema
Dataset 1 (synthetic, clean)	95	93.33	99.5	100	96.85
Dataset 2 (synthetic, messy)	90	58.33	100	100	85.50
Dataset 3 (synthetic, large)	90	99.39	97.5	100	97.07
Dataset 4 (real, Spesa)	45	87.36	75	100	77.71
Dataset 5 (real, Attivazioni)	25	87.56	65	100	70.77



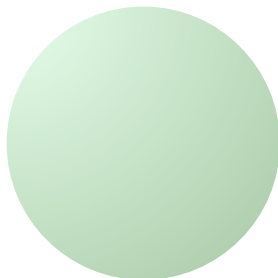
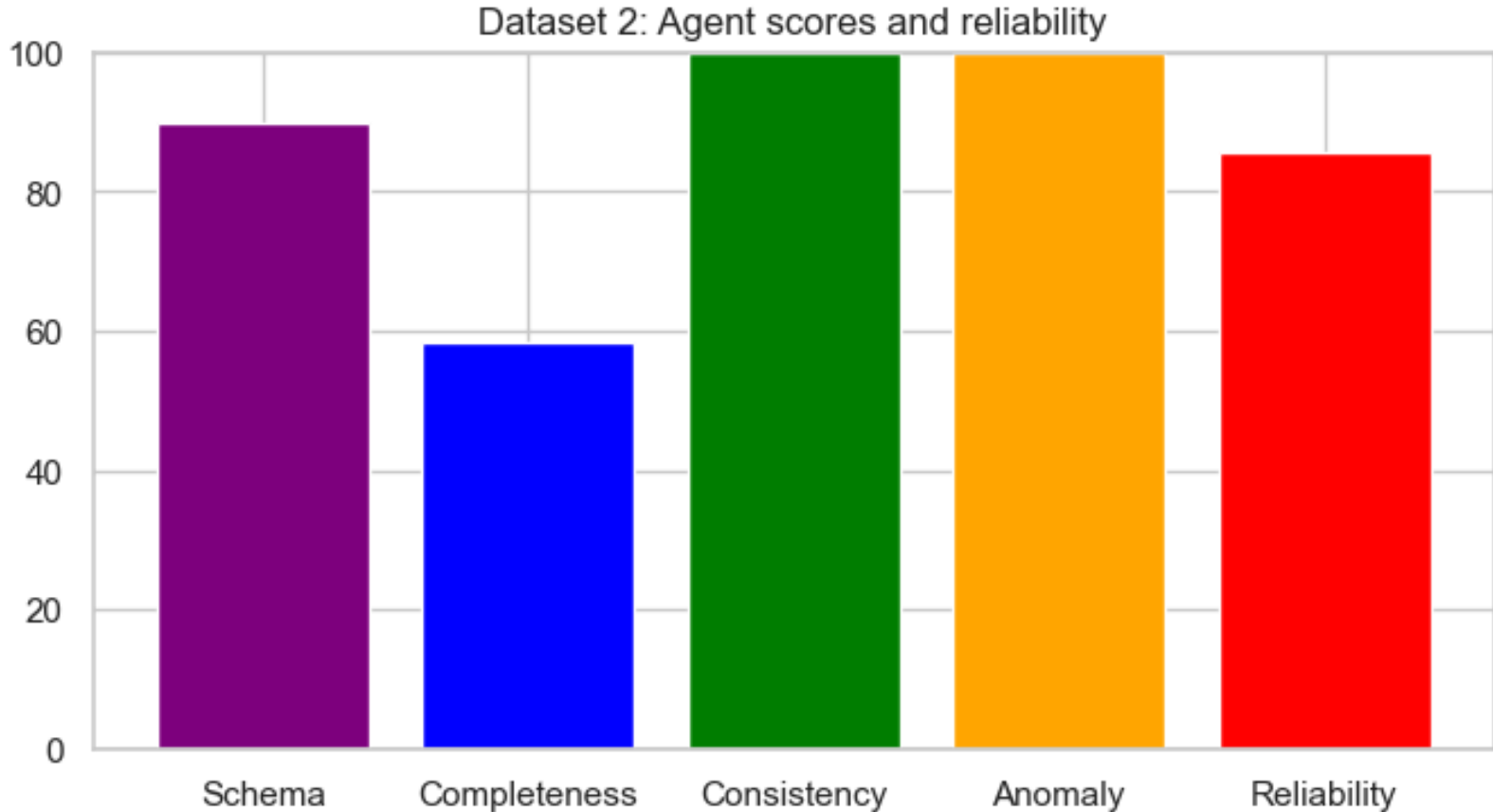
MISSING VALUES PER COLUMN

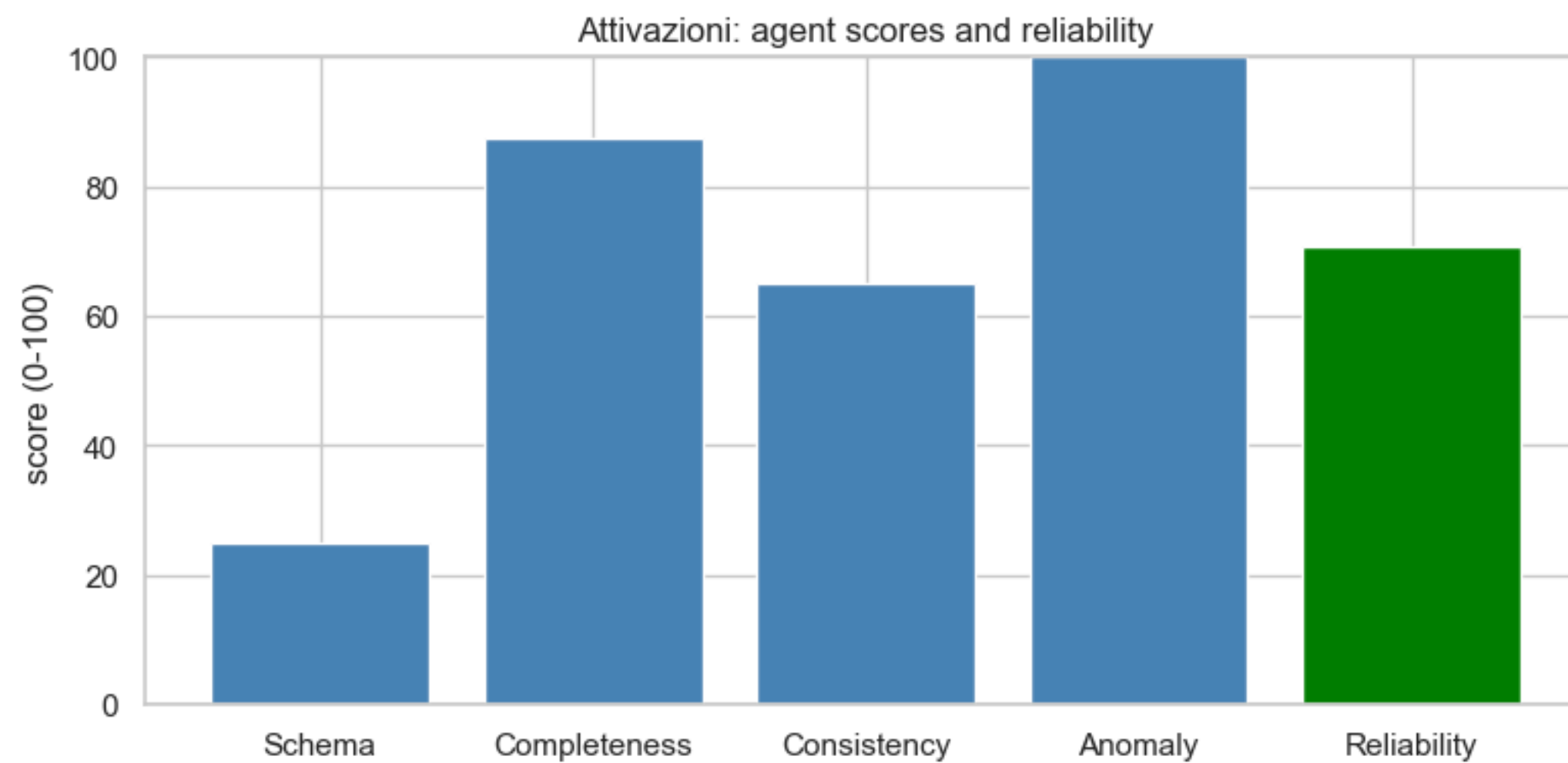
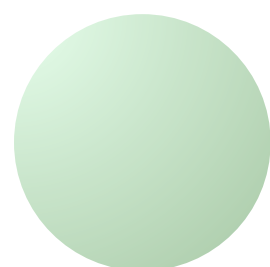
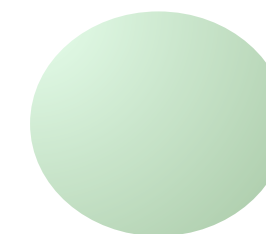




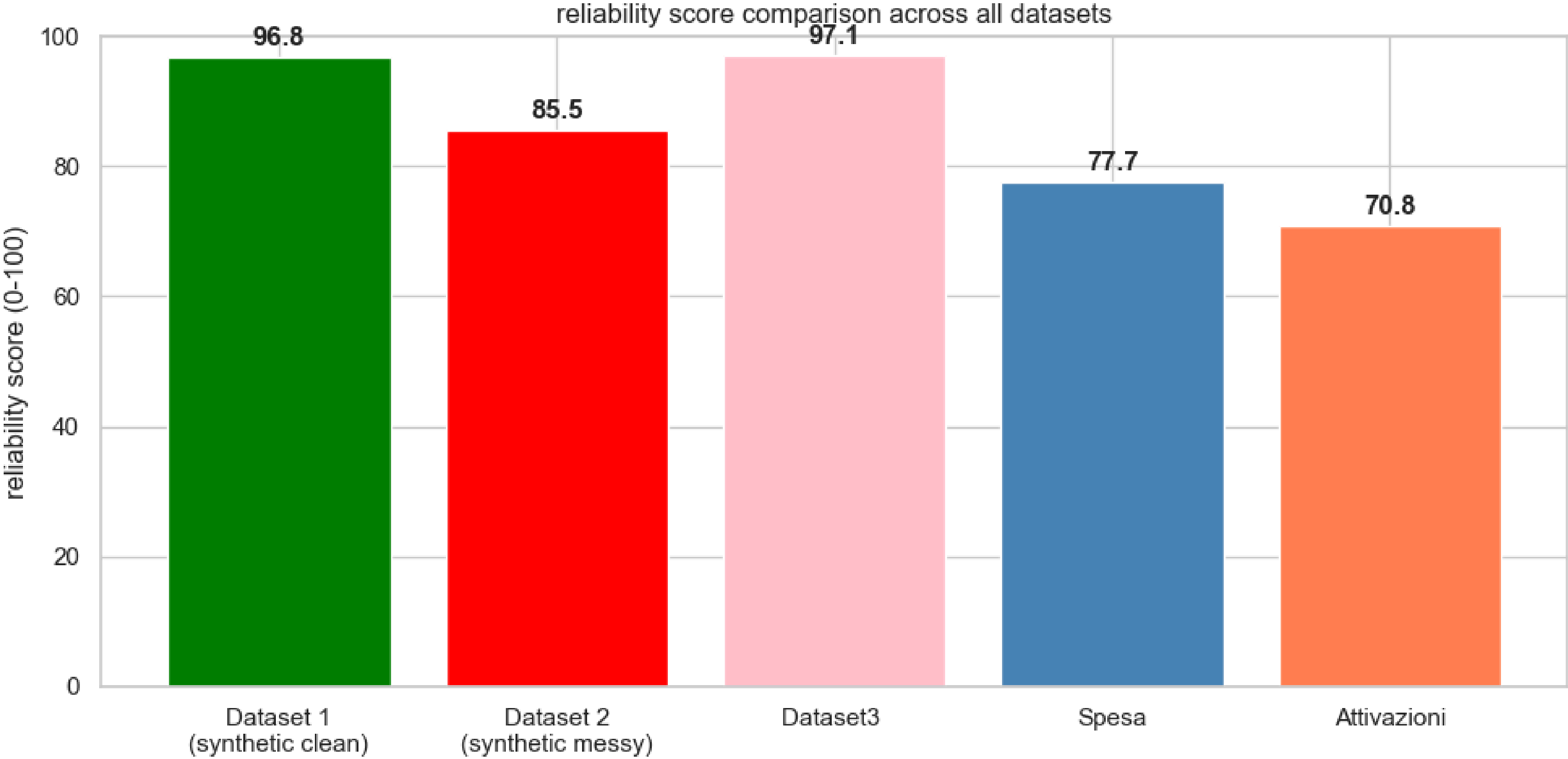


RELIABILITY SCORE BREAKDOWN PER AGENT





OVERALL COMPARISON



CONCLUSION

This project successfully built a modular **multi-agent data quality system** entirely in Python, without requiring any external services to function. The system automatically detects, reports, and fixes data quality issues across datasets of different sizes and structures. Testing on both synthetic and real NoiPA datasets confirmed that the agents behave correctly: clean datasets receive high reliability scores, while datasets with genuine problems receive appropriately low scores with detailed suggestions for improvement.

The key takeaway is that a multi-agent architecture is well-suited for data quality tasks because each type of check (schema, completeness, consistency, anomaly) is independent and can be developed, tested, and improved in isolation without affecting the others. The modular design also makes it straightforward to add new agents in the future.

Several limitations remain. The statistical checks are relatively simple: Z-scores assume normally distributed data, which may not hold for government spending figures. The cross-column validation rules are currently limited to the Rata column format and could be extended to cover more domain-specific logic. The categorical anomaly detection flags a very large number of rare values in real datasets, which may include false positives. . **Future work** could include a more sophisticated anomaly detection methods such as Isolation Forest, deeper LLM integration for automatic schema inference, and support for non-CSV formats such as JSON and relational databases.

STREAMLIT DEMO

To run the interactive app:

- **Clone the repository**
- **Install dependencies: `pip install -r requirements.txt`**
- **Run the app: `python -m streamlit run app.py`**
- **Open the browser at: <http://localhost:8501>**

Upload any CSV file to analyze data quality.



The background features several overlapping circles in various shades of green, ranging from light mint to a darker forest green. The circles are of different diameters, creating a layered, organic feel. The text 'THANK YOU!' is centered horizontally and partially overlaid by these circles.

THANK YOU!